

Genetic Algorithms

Question Bank

Optimization Techniques • CSE(DS)-5003 • Module 3

Dr. Kanchan Rajwar

Department of Mathematics

Indian Institute of Information Technology, Bhopal, India

kanchan.rajwar@iiitbhopal.ac.in

Note to Students

This question bank is based entirely on the course notes *Genetic Algorithms: A Simple Introduction* distributed in class for Optimization Techniques (CSE(DS)-5003), Module 3. All definitions, examples, notation, and numerical values used in the questions are drawn directly from those notes.

The running example ($f(x) = x^2$, $x \in [0, 10]$, $L = 4$ bits, $N = 4$ chromosomes) that appears throughout the questions is the same example developed step by step in the notes. Students are strongly advised to work through the notes carefully before attempting the questions.

The question bank is intended for self-study and exam preparation. It is **not** a substitute for attending class or reading the notes.

Contents

1 Multiple Choice Questions (MCQ)	2
2 Short Answer Questions	20
3 Broad Questions	21
Answer Key	22

Multiple Choice Questions (MCQ)

Choose the single best answer for each question.

Introduction and Motivation

Q1. A Genetic Algorithm (GA) is best described as:

- (a) A deterministic gradient-based method
- (b) A population-based metaheuristic inspired by natural selection
- (c) An exact algorithm that guarantees the global optimum
- (d) A method that works only for continuous functions

Q2. Which of the following problems is *least* suited to a GA?

- (a) A smooth, convex function with an easily computed gradient
- (b) A multimodal black-box function
- (c) The Travelling Salesman Problem
- (d) A discrete scheduling problem

Q3. Classical gradient-based methods fail on NP-hard problems primarily because:

- (a) They require a population of solutions
- (b) The number of possible solutions grows combinatorially with problem size
- (c) They use crossover instead of mutation
- (d) They cannot handle continuous variables

Q4. A *black-box* problem is one where:

- (a) The objective function has many local optima
- (b) No closed-form expression for the objective is available
- (c) The decision variables must be integers
- (d) The search space is infinite

Q5. Which of the following is a classic example of an NP-hard problem?

- (a) Maximising $f(x) = x^2$ over a real interval
- (b) Solving a system of linear equations
- (c) The Travelling Salesman Problem (TSP)
- (d) Computing the derivative of a polynomial

Q6. Darwin's principle of natural selection is best summarised as:

- (a) The fittest individual reproduces without variation
- (b) Fitter individuals are more likely to survive and reproduce
- (c) All individuals reproduce equally
- (d) Only mutated individuals survive

Q7. The key advantage of a GA over gradient-based methods is that it:

- (a) Always finds the global optimum
- (b) Requires no fitness function
- (c) Needs no gradient or analytical formula
- (d) Works only on smooth functions

Q8. A *multimodal* problem is one where the objective function:

- (a) Has a single global optimum
- (b) Has multiple local optima
- (c) Is not defined on a continuous domain
- (d) Cannot be evaluated numerically

Q9. Which of the following best describes a *discrete* optimisation problem?

- (a) The objective function is differentiable everywhere
- (b) The decision variable is a choice, sequence, or permutation
- (c) The search space is unbounded
- (d) The objective function has a unique minimum

Q10. In the context of GAs, “exploration” refers to:

- (a) Refining already-good solutions using selection
- (b) Searching new, unexplored regions of the search space
- (c) Computing the fitness of each chromosome
- (d) Decoding the genotype to a phenotype

Basic Terms

Q11. A *chromosome* in a GA represents:

- (a) A single gene
- (b) A single candidate solution in encoded form
- (c) The entire population
- (d) The fitness function

Q12. In a binary-encoded GA, a *gene* is:

- (a) The entire chromosome
- (b) A single bit (0 or 1)
- (c) The decoded phenotype
- (d) The fitness value

Q13. The *population* in a GA refers to:

- (a) The best chromosome found so far
- (b) The set of all genes in one chromosome
- (c) The complete set of chromosomes maintained at any point
- (d) The number of generations completed

Q14. The *genotype* is:

- (a) The decoded real-world value of a solution
- (b) The encoded binary string the GA works with directly
- (c) The fitness score of a chromosome
- (d) The number of bits in a chromosome

Q15. The *phenotype* is:

- (a) The binary string stored internally by the GA
- (b) The actual solution value in the problem domain
- (c) The selection probability of a chromosome
- (d) The mutation rate applied to a chromosome

Q16. For a 4-bit binary chromosome 1011, the decoded integer value is:

- (a) 9
- (b) 10
- (c) 11
- (d) 12

Q17. For $x \in [0, 10]$ with $L = 4$ bits ($2^4 - 1 = 15$), the chromosome 1100 decodes to phenotype $x =$:

- (a) 6.67
- (b) 7.33
- (c) 8.00

- (d) 9.00
- Q18.** One complete cycle of selection, crossover, mutation, and replacement is called a:
- (a) Gene
 - (b) Generation
 - (c) Population
 - (d) Schema
- Q19.** The fitness of a chromosome is:
- (a) Always equal to the objective function value
 - (b) A scalar measure of how good a solution is
 - (c) The number of bits that were mutated
 - (d) The rank of the chromosome in the population
- Q20.** For a minimisation problem, the fitness transformation $F(x) = 1/(1 + f(x))$ ensures that:
- (a) Larger $f(x)$ gives larger fitness
 - (b) Smaller $f(x)$ gives larger fitness
 - (c) All chromosomes receive equal fitness
 - (d) Fitness is always negative
- Q21.** Unlike classical methods such as Newton–Raphson, a GA:
- (a) Refines a single solution iteratively
 - (b) Evolves an entire population simultaneously
 - (c) Requires the objective function to be differentiable
 - (d) Always converges to the global optimum
- Q22.** The chromosome 0110 with $x \in [0, 10]$, $L = 4$ decodes to phenotype $x =$:
- (a) 3.33
 - (b) 4.00
 - (c) 5.00
 - (d) 6.00
- Q23.** In a GA, fitness is evaluated on the:
- (a) Genotype directly
 - (b) Decoded phenotype

- (c) Mating pool
- (d) Cumulative probability

Encoding

Q24. With $L = 5$ bits, the number of distinct binary strings is:

- (a) 16
- (b) 31
- (c) 32
- (d) 64

Q25. The denominator $2^L - 1$ in the decoding formula represents:

- (a) The number of distinct strings
- (b) The number of equally spaced intervals between $x_{\min}$ and $x_{\max}$
- (c) The mutation rate
- (d) The population size

Q26. The *precision* of a binary encoding is defined as:

- (a) $x_{\max} - x_{\min}$
- (b) $(x_{\max} - x_{\min})/2^L$
- (c) $(x_{\max} - x_{\min})/(2^L - 1)$
- (d) $2^L - 1$

Q27. To achieve a precision of $\varepsilon = 0.01$ for $x \in [0, 10]$, the minimum number of bits required is:

- (a) 8
- (b) 9
- (c) 10
- (d) 11

Q28. Permutation encoding is used instead of binary encoding when:

- (a) The variable is continuous and real-valued
- (b) The solution is an ordering of elements, such as in TSP
- (c) Higher precision is required
- (d) The population size is large

Q29. For $x \in [2, 8]$ with $L = 4$ bits, the chromosome 1011 (integer value 11) decodes to:

- (a) $x = 4.4$
- (b) $x = 6.4$
- (c) $x = 7.4$
- (d) $x = 5.4$

Q30. Real-coded encoding is preferred over binary encoding when:

- (a) The problem is combinatorial
- (b) High precision for continuous variables is required
- (c) The chromosome length must be minimised
- (d) The problem involves route planning

Q31. For $x \in [0, 10]$ with $L = 4$ bits, the chromosome 0001 decodes to phenotype $x =$:

- (a) 0.33
- (b) 0.67
- (c) 1.00
- (d) 1.33

Q32. A binary chromosome of length $L = 4$ can represent at most how many distinct values of x ?

- (a) 8
- (b) 15
- (c) 16
- (d) 32

Q33. The reason binary encoding is *unsuitable* for the TSP is that:

- (a) Binary strings are too short to encode cities
- (b) A random binary string may repeat or omit cities, giving an invalid route
- (c) Binary encoding cannot represent integers
- (d) The TSP requires real-valued variables

The Fitness Function

Q34. For a maximisation problem, the fitness function $F(x)$ is set to:

- (a) $1/f(x)$
- (b) $C_{\max} - f(x)$

- (c) $f(x)$
- (d) $1/(1 + f(x))$

Q35. Which transformation is *not* a valid way to convert a minimisation problem to a maximisation problem for a GA?

- (a) $F(x) = 1/(1 + f(x))$
- (b) $F(x) = C_{\max} - f(x)$
- (c) $F(x) = -f(x)$
- (d) $F(x) = f(x)^2$

Q36. Which chromosome is most likely to be selected given fitnesses 53.73, 16.00, 64.00, 0.44 for 1011, 0110, 1100, 0001 respectively?

- (a) 1011
- (b) 0110
- (c) 1100
- (d) 0001

Q37. *Fitness scaling* is used primarily to:

- (a) Increase the mutation rate
- (b) Prevent one dominant chromosome from taking over too quickly
- (c) Decrease the population size
- (d) Increase the crossover rate

Q38. For the minimisation example with $F(x) = 1/(1 + f(x))$, the chromosome 0001 ($x = 0.67$, $f = 0.44$) has fitness approximately:

- (a) 0.015
- (b) 0.018
- (c) 0.059
- (d) 0.694

Q39. The objective function $f(x)$ and the fitness function $F(x)$ differ in that:

- (a) $f(x)$ is always larger than $F(x)$
- (b) $F(x)$ is designed so the GA always maximises it, regardless of whether the problem is maximisation or minimisation
- (c) $f(x)$ is used internally by the GA; $F(x)$ is the real-world quantity
- (d) They are always identical

Q40. A safe choice for the constant $C_{\max}$ in $F(x) = C_{\max} - f(x)$ is:

- (a) $C_{\max} = 0$
- (b) $C_{\max} = \min_i f(x_i)$
- (c) $C_{\max} = \max_i f(x_i) + 1$
- (d) $C_{\max} = \bar{f}$

Q41. For the running example ($f(x) = x^2$, $x \in [0, 10]$, $L = 4$), the chromosome 0110 has fitness:

- (a) 53.73
- (b) 16.00
- (c) 64.00
- (d) 0.44

Selection

Q42. In roulette-wheel selection, the selection probability of chromosome i is:

- (a) f_i/N
- (b) $f_i/f_{\max}$
- (c) $f_i/\sum_j f_j$
- (d) $1/N$

Q43. The *expected count* of a chromosome in roulette-wheel selection is:

- (a) $f_i/f_{\max}$
- (b) $f_i/\bar{f}$
- (c) $f_i \times N$
- (d) $p_i \times f_i$

Q44. Using the running example ($\sum f = 134.17$), the selection probability of 1100 ($f = 64.00$) is approximately:

- (a) 0.119
- (b) 0.401
- (c) 0.477
- (d) 0.003

Q45. In roulette-wheel selection with cumulative probabilities $P_1 = 0.401$, $P_2 = 0.520$, $P_3 = 0.997$, $P_4 = 1.000$, a draw of $r = 0.65$ selects chromosome:

- (a) 1 (1011)

- (b) 2 (0110)
- (c) 3 (1100)
- (d) 4 (0001)

Q46. Tournament selection selects a parent by:

- (a) Spinning a fitness-proportionate wheel
- (b) Picking the chromosome with the highest rank
- (c) Drawing a random subset and choosing the fittest in that subset
- (d) Copying the best chromosome directly

Q47. Rank selection is preferred over roulette-wheel selection when:

- (a) The fitness values are all equal
- (b) Fitness values are spread very unevenly, risking one chromosome dominating
- (c) The population size is very small
- (d) The mutation rate is very high

Q48. *Elitism* in a GA refers to:

- (a) Selecting only the top 50% of chromosomes
- (b) Copying the best chromosome directly into the next generation
- (c) Applying crossover only to the fittest pair
- (d) Increasing the mutation rate for weaker chromosomes

Q49. The main risk of elitism is:

- (a) It slows convergence
- (b) It can reduce population diversity and contribute to premature convergence
- (c) It increases the number of fitness evaluations
- (d) It requires a larger population

Q50. In the running example, 1100 is selected twice because:

- (a) It has the longest chromosome
- (b) Its cumulative interval $(0.520, 0.997]$ captures two random draws
- (c) It was chosen by elitism
- (d) It has the highest mutation rate

Q51. In roulette-wheel selection, a draw of $r = 0.21$ selects chromosome 1011 because:

- (a) $P_1 = 0.401 \geq 0.21$ and it is the first chromosome satisfying this

- (b) $P_2 = 0.520 \geq 0.21$
- (c) 1011 has the highest fitness
- (d) $r < p_1$

Q52. The total of all selection probabilities $\sum_i p_i$ must equal:

- (a) N (population size)
- (b) 0
- (c) 1
- (d) $\bar{f}$ (average fitness)

Q53. In the running example, chromosome 0001 is not selected because:

- (a) It has zero fitness
- (b) Its selection probability is nearly zero and no random draw landed in its tiny interval
- (c) It was removed by elitism
- (d) Its cumulative probability exceeds 1

Crossover

Q54. In single-point crossover with $L = 4$, the cut point is chosen uniformly at random from:

- (a) Positions 0 to 4
- (b) Positions 1 to 3
- (c) Positions 1 to 4
- (d) Positions 0 to 3

Q55. Applying single-point crossover to 10|11 and 11|00 (cut after position 2) produces:

- (a) 1011 and 1100
- (b) 1000 and 1111
- (c) 1010 and 1101
- (d) 0011 and 1100

Q56. The crossover rate p_c is typically:

- (a) Very low, between 0.001 and 0.05
- (b) Moderate, around 0.5
- (c) High, between 0.6 and 0.9

(d) Always exactly 1.0

Q57. When $r = 0.91 > p_c = 0.8$ for a pair, the GA:

- (a) Applies crossover at a random cut point
- (b) Applies mutation instead
- (c) Copies the parents unchanged into the next generation
- (d) Discards both parents

Q58. Two-point crossover differs from single-point crossover in that:

- (a) It uses three parents
- (b) It swaps the middle segment between two cut points
- (c) It flips individual bits
- (d) It uses a coin toss for each bit position

Q59. Uniform crossover decides which parent contributes each bit by:

- (a) Always taking from the fitter parent
- (b) Choosing the parent with the higher fitness
- (c) A fair coin toss at each bit position
- (d) A single cut point

Q60. Crossover alone *cannot* introduce a gene that is:

- (a) Present in both parents
- (b) Present in one parent only
- (c) Absent from every chromosome in the population
- (d) Present in the fittest chromosome

Q61. After single-point crossover of 1011 and 1100 (cut after position 2), offspring 1111 has fitness $f(x) = x^2$ equal to:

- (a) 64.00
- (b) 53.73
- (c) 100.00
- (d) 28.44

Q62. In Pair 2 of the running example ($r = 0.91 > 0.8$), the offspring are:

- (a) 1000 and 1111
- (b) 1010 and 1100

- (c) 1011 and 1100 (unchanged copies of parents)
- (d) 1111 and 1011

Q63. A high crossover rate p_c encourages:

- (a) More mutations per generation
- (b) Rapid spread of useful genes across the population
- (c) Slower convergence
- (d) Larger population size

Q64. After crossover, the new population in the running example has average fitness:

- (a) 33.54
- (b) 59.36
- (c) 61.54
- (d) 65.56

Mutation

Q65. In bit-flip mutation, each bit is flipped with probability:

- (a) p_c
- (b) p_m
- (c) $1/N$
- (d) $1/2^L$

Q66. The mutation rate p_m is typically:

- (a) High, between 0.6 and 0.9
- (b) Moderate, around 0.5
- (c) Small, between 0.001 and 0.05
- (d) Always exactly 0

Q67. A common rule of thumb for setting p_m is:

- (a) $p_m = p_c$
- (b) $p_m = 1/L$
- (c) $p_m = 1/N$
- (d) $p_m = L/N$

Q68. Flipping bit 3 of 1000 gives:

- (a) 1100
- (b) 1001
- (c) 1010
- (d) 0000

Q69. The primary role of mutation in a GA is to:

- (a) Recombine existing genes from two parents
- (b) Introduce entirely new genetic material not present in the population
- (c) Select the fittest chromosomes
- (d) Decode the genotype to a phenotype

Q70. If the mutation rate is too high, the GA:

- (a) Converges too quickly
- (b) Behaves like a random walk and destroys good solutions
- (c) Never finds any improvement
- (d) Stops after one generation

Q71. If the mutation rate is too low, the GA:

- (a) Explores too widely
- (b) Loses diversity and may stagnate
- (c) Finds the global optimum faster
- (d) Applies crossover more frequently

Q72. After mutating 1000 to 1010 ($x \in [0, 10]$, $L = 4$), the new fitness $f(x) = x^2$ is approximately:

- (a) 28.44
- (b) 36.00
- (c) 44.49
- (d) 53.73

Q73. Crossover and mutation are complementary because:

- (a) Both flip individual bits
- (b) Crossover exploits existing genes; mutation explores new ones
- (c) Both require a mating pool
- (d) Crossover creates new genes; mutation recombines old ones

- Q74.** In the running example with $p_m = 0.1$, a bit is flipped only if:
- (a) The chromosome has the lowest fitness
 - (b) A random draw $r \leq 0.1$ for that bit
 - (c) The crossover step was skipped
 - (d) The generation number is odd
- Q75.** For a chromosome of length $L = 4$ with $p_m = 0.1$, the *expected* number of bits flipped per chromosome is:
- (a) 0.1
 - (b) 0.4
 - (c) 1.0
 - (d) 4.0

A Complete GA Run

- Q76.** In the running example, the initial average fitness of the population $\{1011, 0110, 1100, 0001\}$ is:
- (a) 59.36
 - (b) 33.54
 - (c) 61.54
 - (d) 65.56
- Q77.** After selection, the average fitness of the mating pool $\{1011, 1100, 1011, 1100\}$ is:
- (a) 33.54
 - (b) 61.54
 - (c) 59.36
 - (d) 65.56
- Q78.** After crossover, the best fitness in the population is:
- (a) 64.00
 - (b) 53.73
 - (c) 100.00
 - (d) 44.49

- Q79.** The chromosome that achieves the global optimum $f = 100$ is:
- (a) 1011

- (b) 1100
- (c) 1010
- (d) 1111

Q80. After mutation, the average fitness of the final population is:

- (a) 61.54
- (b) 59.36
- (c) 65.56
- (d) 33.54

Q81. The average fitness after selection (59.36) is higher than the initial average (33.54) because:

- (a) Crossover was applied
- (b) Mutation was applied
- (c) Weaker chromosomes were not selected into the mating pool
- (d) The chromosome length increased

Q82. In the running example, the GA finds the global optimum $f = 100$ during which step?

- (a) Initialisation
- (b) Selection
- (c) Crossover
- (d) Mutation

Q83. After mutation, chromosome 1000 becomes 1010. Its fitness improves from:

- (a) 53.73 to 64.00
- (b) 28.44 to 44.49
- (c) 16.00 to 53.73
- (d) 0.44 to 16.00

Q84. The best fitness *does not change* between the crossover and mutation steps in the running example because:

- (a) Mutation always worsens fitness
- (b) The best chromosome 1111 ($f = 100$) was not mutated
- (c) Selection removed the best chromosome
- (d) Crossover was not applied in generation 1

Q85. Across one generation of the running example, the average fitness improved from 33.54 to 65.56, an increase of approximately:

- (a) 25%
- (b) 50%
- (c) 95%
- (d) 200%

Parameters and Practical Guidelines

Q86. The total number of fitness evaluations in a GA run is approximately:

- (a) $N + T$
- (b) $N \times T$
- (c) N/T
- (d) T/N

Q87. *Premature convergence* in a GA occurs when:

- (a) The mutation rate is too high
- (b) The population becomes nearly identical before a good solution is found
- (c) The crossover rate is too low
- (d) The chromosome length is too short

Q88. Which of the following is *not* a remedy for premature convergence?

- (a) Increasing p_m
- (b) Using tournament or rank selection
- (c) Decreasing the population size N
- (d) Restarting with a fresh sub-population

Q89. The chromosome length L is determined by:

- (a) The population size N
- (b) The required precision ε via $2^L - 1 \geq (x_{\max} - x_{\min})/\varepsilon$
- (c) The crossover rate p_c
- (d) The number of generations T

Q90. Which parameter controls how long a GA runs?

- (a) N (population size)
- (b) p_c (crossover rate)

- (c) T (number of generations)
- (d) L (chromosome length)

Q91. A typical value for the crossover rate p_c is:

- (a) 0.01
- (b) 0.50
- (c) 0.80
- (d) 1.00

Q92. A typical value for the mutation rate p_m is:

- (a) 0.80
- (b) 0.50
- (c) 0.01
- (d) 1.00

Q93. Once premature convergence occurs (all chromosomes identical), which operator can still drive further improvement?

- (a) Selection
- (b) Crossover
- (c) Mutation
- (d) Elitism

Advantages and Limitations

Q94. A GA cannot guarantee:

- (a) Finding a good solution
- (b) That the global optimum has been found
- (c) That fitness evaluations are performed
- (d) That crossover is applied

Q95. A GA is well-suited to parallel computing because:

- (a) It uses a single solution at a time
- (b) The entire population's fitness can be evaluated simultaneously
- (c) Crossover requires only one processor
- (d) Mutation is deterministic

Q96. The computational cost of a GA scales with:

- (a) $N + T$
- (b) $N \times T$ fitness evaluations
- (c) $L \times p_c$
- (d) $p_m \times N$

Q97. Which statement about GAs is *false*?

- (a) A GA requires only a fitness function, not a gradient
- (b) A GA can handle discrete, continuous, and black-box problems
- (c) A GA always finds the exact global optimum
- (d) A GA is a population-based metaheuristic

Q98. The *broad applicability* of a GA means that:

- (a) The same core loop applies to many problem types with only the encoding and fitness function changing
- (b) A GA can be applied without any problem-specific knowledge
- (c) A GA requires no parameter tuning
- (d) A GA always outperforms exact methods

Q99. *Parameter sensitivity* as a limitation of GAs means that:

- (a) GAs cannot be applied to sensitive problems
- (b) Poor choices of N , T , p_c , p_m can lead to poor performance, requiring experimentation to tune them
- (c) GAs are sensitive to the initial random seed only
- (d) GAs cannot handle sensitive fitness functions

Q100. When is a GA *not* the preferred choice?

- (a) When the objective function is non-differentiable
- (b) When the problem is NP-hard
- (c) When the objective is smooth, convex, and a proven optimal solution is required
- (d) When the search space is multimodal

Short Answer Questions

Answer each question in 4–6 sentences.

- S1.** Explain the difference between genotype and phenotype in a GA, and describe how a binary chromosome is decoded to a real-valued phenotype for $x \in [x_{\min}, x_{\max}]$.
- S2.** Describe roulette-wheel selection. Why is chromosome 1100 selected twice in the running example?
- S3.** Explain the roles of crossover and mutation in a GA, and why both operators are necessary.
- S4.** What is premature convergence, and what strategies can be used to prevent it?
- S5.** State the five primary parameters of a GA and give the typical range for each. What determines the chromosome length L ?

Broad Questions

Answer each question in detail, with definitions, derivations, and examples where appropriate.

- B1.** Describe the complete working of a Genetic Algorithm. Include the six-step loop, the role of each genetic operator, the stopping criteria, and a worked example tracing one full generation for $f(x) = x^2$ over $x \in [0, 10]$ with $L = 4$ bits and $N = 4$ chromosomes.
- B2.** Explain binary encoding and decoding in a GA. Define precision, derive the formula for choosing the chromosome length L , and give a fully worked numerical example.
- B3.** Discuss the advantages and limitations of Genetic Algorithms. Under what conditions would you prefer a GA over a classical gradient-based method, and when would you not?
- B4.** Explain roulette-wheel selection in detail. Derive the selection probability and cumulative probability for the running example ($f(x) = x^2$, $x \in [0, 10]$, $L = 4$), and trace four selection draws to determine the mating pool. Why does the chromosome 1100 get selected twice?
- B5.** Compare and contrast single-point crossover, two-point crossover, and uniform crossover. For each, state how the offspring are produced, when it is preferred, and give a brief example using the chromosomes 1011 and 1100.

Answer Key

MCQ Answers

Q	Topic	Ans	Q	Topic	Ans	Q	Topic	Ans	Q	Topic	Ans
1	Introduction	(b)	26	Encoding	(b)	51	Selection	(a)	76	Complete run	(b)
2	Introduction	(a)	27	Encoding	(c)	52	Selection	(b)	77	Complete run	(c)
3	Introduction	(b)	28	Encoding	(c)	53	Selection	(b)	78	Complete run	(c)
4	Introduction	(b)	29	Encoding	(b)	54	Crossover	(b)	79	Complete run	(d)
5	Introduction	(c)	30	Encoding	(b)	55	Crossover	(b)	80	Complete run	(c)
6	Introduction	(b)	31	Encoding	(c)	56	Crossover	(c)	81	Complete run	(c)
7	Introduction	(c)	32	Encoding	(b)	57	Crossover	(c)	82	Complete run	(c)
8	Introduction	(b)	33	Encoding	(b)	58	Crossover	(b)	83	Complete run	(b)
9	Introduction	(b)	34	Fitness	(c)	59	Crossover	(b)	84	Complete run	(b)
10	Introduction	(b)	35	Fitness	(d)	60	Crossover	(c)	85	Complete run	(c)
11	Basic terms	(b)	36	Fitness	(c)	61	Crossover	(c)	86	Parameters	(b)
12	Basic terms	(b)	37	Fitness	(b)	62	Crossover	(b)	87	Parameters	(b)
13	Basic terms	(c)	38	Fitness	(b)	63	Crossover	(c)	88	Parameters	(c)
14	Basic terms	(b)	39	Fitness	(d)	64	Crossover	(c)	89	Parameters	(b)
15	Basic terms	(b)	40	Fitness	(b)	65	Mutation	(b)	90	Parameters	(c)
16	Basic terms	(c)	41	Fitness	(c)	66	Mutation	(c)	91	Parameters	(c)
17	Basic terms	(b)	42	Selection	(c)	67	Mutation	(b)	92	Parameters	(c)
18	Basic terms	(b)	43	Selection	(b)	68	Mutation	(b)	93	Parameters	(c)
19	Basic terms	(b)	44	Selection	(c)	69	Mutation	(c)	94	Adv & Lim	(b)
20	Basic terms	(b)	45	Selection	(c)	70	Mutation	(b)	95	Adv & Lim	(b)
21	Basic terms	(b)	46	Selection	(b)	71	Mutation	(b)	96	Adv & Lim	(b)
22	Basic terms	(b)	47	Selection	(b)	72	Mutation	(c)	97	Adv & Lim	(c)
23	Basic terms	(b)	48	Selection	(b)	73	Mutation	(b)	98	Adv & Lim	(a)
24	Encoding	(c)	49	Selection	(a)	74	Mutation	(b)	99	Adv & Lim	(b)
25	Encoding	(b)	50	Selection	(c)	75	Mutation	(b)	100	Adv & Lim	(c)

Short Answer Model Answers

S1. Answer: The *genotype* is the encoded binary string that the GA operates on internally (e.g. 1011). The *phenotype* is the actual real-world value the string represents after decoding. Decoding proceeds in two steps: first, the binary string is converted to an integer by summing its place values; second, this integer is scaled to the target range using

$$x = x_{\min} + \frac{x_{\max} - x_{\min}}{2^L - 1} \times (\text{integer value}).$$

For example, 1011 decodes to integer 11, and for $x \in [0, 10]$ with $L = 4$, this gives $x = (10/15) \times 11 \approx 7.33$. The GA always works on genotypes, but fitness is evaluated on the decoded phenotype.

S2. Answer: In roulette-wheel selection, each chromosome is assigned a slice of a wheel proportional to its fitness. A uniform random number $r \in [0, 1]$ is drawn; the chromosome whose cumulative probability interval $[P_{i-1}, P_i]$ contains r is selected. In the running example, 1100 has fitness 64.00 out of a total of 134.17, giving selection probability $p_3 \approx 0.477$ and cumulative interval $(0.520, 0.997]$. Two of the four random draws ($r = 0.65$ and $r = 0.94$) fall inside this interval, so 1100 is selected twice, consistent with its expected count of $64.00/33.54 \approx 1.91 \approx 2$.

S3. Answer: Crossover recombines two parent chromosomes by swapping portions after a randomly chosen cut point, producing offspring that may inherit the best traits of both parents. Mutation randomly flips individual bits with a small probability p_m , introducing genetic material that may not exist anywhere in the current population. Both operators are necessary because crossover alone can only recombine genes already present — if a useful gene is absent from every chromosome, crossover cannot introduce it. Mutation is the only source of entirely new genetic material. Together, crossover exploits good existing genes while mutation explores new regions of the search space.

S4. Answer: Premature convergence occurs when the population becomes nearly identical before a satisfactory solution has been found. Once all chromosomes are the same, crossover produces no new solutions and the search stalls, often at a local optimum. Strategies to prevent it include: increasing the mutation rate p_m to restore diversity; using tournament or rank selection to moderate selection pressure; applying elitism carefully (preserving only one or two elite chromosomes); and restarting part of the population with randomly generated chromosomes if no improvement is observed over many generations.

S5. Answer: The five primary parameters are: population size N (20–200), number of generations T (100–1000), crossover rate p_c (0.6–0.9), mutation rate p_m (0.001–0.05), and chromosome length L (determined by the required precision). The chromosome length is set by the desired precision ε : choose L such that $2^L - 1 \geq (x_{\max} - x_{\min})/\varepsilon$. For example, for $x \in [0, 10]$ with $\varepsilon = 0.01$, we need $2^L \geq 1001$, giving $L = 10$ bits. The total computational cost of a GA run is approximately $N \times T$ fitness evaluations.

Broad Question Model Answers

B1. Answer: A GA maintains a population of N encoded candidate solutions and improves them over successive generations using the following loop.

Step 1 — Initialise. Generate a random initial population. Example: $\{1011, 0110, 1100, 0001\}$ with fitnesses $\{53.73, 16.00, 64.00, 0.44\}$ and average fitness 33.54.

Step 2 — Evaluate. Compute $F(x) = f(x)$ for each chromosome using the two-step decode (binary $\rightarrow$ integer $\rightarrow$ phenotype).

Step 3 — Select. Apply roulette-wheel selection. With draws $r = 0.21, 0.65, 0.38, 0.94$, the mating pool is $\{1011, 1100, 1011, 1100\}$ (average fitness 59.36).

Step 4 — Crossover. With $p_c = 0.8$: Pair 1 ($r = 0.43 \leq 0.8$) crosses 10|11 and 11|00 to produce 1000 and 1111. Pair 2 ($r = 0.91 > 0.8$) is copied unchanged. Average fitness after crossover: 61.54, best fitness: 100.00.

Step 5 — Mutate. With $p_m = 0.1$: bit 3 of 1000 is flipped to give 1010. Final population: $\{1010, 1111, 1011, 1100\}$, average fitness 65.56.

Step 6 — Replace. The new population becomes the current population and the loop returns to Step 2. The GA stops when a fixed T is reached, when the best fitness does not improve for many consecutive generations, or when an acceptable solution is found.

B2. Answer: Binary encoding represents each candidate solution as a string of L bits, giving 2^L distinct strings. Decoding proceeds in two steps.

Step 1 — Integer decoding. Sum the place values: for 1011, $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11$.

Step 2 — Range scaling.

$$x = x_{\min} + \frac{x_{\max} - x_{\min}}{2^L - 1} \times (\text{integer}).$$

The denominator $2^L - 1$ is the number of equally spaced intervals between $x_{\min}$ and $x_{\max}$.

Precision. $\delta = (x_{\max} - x_{\min}) / (2^L - 1)$. To achieve desired precision ε , choose L such that $2^L - 1 \geq (x_{\max} - x_{\min}) / \varepsilon$.

Worked example. For $x \in [0, 10]$, $\varepsilon = 0.01$: $2^L - 1 \geq 1000 \Rightarrow 2^L \geq 1001$. Since $2^9 = 512 < 1001$ and $2^{10} = 1024 \geq 1001$, we need $L = 10$ bits with achieved precision $10/1023 \approx 0.0098$. For the running example ($L = 4$, $x \in [0, 10]$): $1100 \rightarrow$ integer 12 $\rightarrow x = (10/15) \times 12 = 8.00 \rightarrow f(8.00) = 64.00$.

B3. Answer: *Advantages.* (i) *No gradient required:* applicable to black-box, non-differentiable, and discontinuous problems. (ii) *Broad applicability:* the same core loop handles discrete, continuous, multimodal, and NP-hard problems. (iii) *Global search:* less likely to get trapped in a local optimum than single-trajectory gradient

methods. (iv) *Parallelism*: all fitness evaluations in a generation are independent and can be distributed across processors.

Limitations. (i) *No optimality guarantee*: cannot certify that the global optimum has been found. (ii) *Parameter sensitivity*: N , T , p_c , p_m , and encoding must be tuned by experimentation. (iii) *Computational cost*: scales with $N \times T$ fitness evaluations, prohibitive when each evaluation is expensive.

When to prefer a GA: non-differentiable, multimodal, discrete, or black-box objective; near-optimal solution acceptable; parallel hardware available.

When not to prefer a GA: smooth, convex, differentiable objective (gradient descent is faster and exact); proven optimality required; fitness evaluations are very expensive.

B4. Answer: In roulette-wheel selection, each chromosome i receives selection probability $p_i = f_i / \sum_j f_j$ and cumulative probability $P_i = \sum_{k=1}^i p_k$. A uniform random draw r selects the first chromosome i for which $P_i \geq r$.

Running example. Total fitness = 134.17.

Chr.	f_i	p_i	P_i	Exp. count
1011	53.73	0.401	0.401	1.60
0110	16.00	0.119	0.520	0.48
1100	64.00	0.477	0.997	1.91
0001	0.44	0.003	1.000	0.01

Four draws: $r = 0.21 \rightarrow 1011$; $r = 0.65 \rightarrow 1100$; $r = 0.38 \rightarrow 1011$; $r = 0.94 \rightarrow 1100$.

Mating pool: $\{1011, 1100, 1011, 1100\}$. 1100 is selected twice because its slice $(0.520, 0.997]$ spans 0.477 of the wheel and both $r = 0.65$ and $r = 0.94$ fall within it, consistent with expected count ≈ 1.91 .

B5. Answer: *Single-point crossover*. One cut point chosen uniformly from positions 1 to $L - 1$; tails are exchanged. Example (cut after position 2): 10|11 and 11|00 $\rightarrow$ 1000 and 1111. Simple and widely used; preserves long building blocks near the chromosome ends.

Two-point crossover. Two cut points chosen; the middle segment is swapped. Example (cuts after positions 1 and 3): 1|01|1 and 1|10|0 $\rightarrow$ 1101 and 1010. Preferred when preserving both ends of the chromosome is important.

Uniform crossover. A fair coin toss at each bit position decides which parent contributes that bit. Produces the most thorough mixing; does not favour any region of the chromosome. Preferred when no region of the chromosome is more important than another.

Comparison. As mixing increases from single-point to uniform, disruption of existing good partial solutions also increases. The choice depends on the structure of the problem: if genes at certain positions are strongly linked, point-based crossover is preferable; if genes are largely independent, uniform crossover is more effective.